mathematics

MDPI

Article

Algorithm for Generating Bifurcation Diagrams Using Poincaré Intersection Plane

Luis Javier Ontañón-García [1], Juan Gonzalo Barajas-Ramírez [2], Eric Campos-Cantón [2], Daniel Alejandro Magallón-García [1], César Arturo Guerra-García [1], José Ricardo Cuesta-García [3], Jonatan Pena-Ramirez [3,*] and José Luis Echenausía-Monroy [3,*]

[1] Coordinación Académica Región Altiplano Oeste, Universidad Autónoma de San Luis Potosí, Carretera a Santo Domingo 200, Salinas de Hidalgo 78600, SLP, Mexico; luis.ontanon@uaslp.mx (L.J.O.-G.); daniel.magallon6532@academicos.udg.mx (D.A.M.-G.); cesar.guerra@uaslp.mx (C.A.G.-G.)

[2] División de Control y Sistemas Dinámicos, Instituto Potosino de Investigación Científica y Tecnológica A.C. (IPICyT), Camino a la Presa San José 2255, Lomas 4ta. Sección, San Luis Potosí 78216, SLP, Mexico; jgbarajas@ipicyt.edu.mx (J.G.B.-R.); eric.campos@ipicyt.edu.mx (E.C.-C.)

[3] Applied Physics Division, Department of Electronics and Telecommunications, CICESE Research Center, Carr. Ensenada-Tijuana 3918, Zona Playitas, Ensenada, Ensenada 22860, BC, Mexico; jcuesta@cicese.mx

* Correspondence: jpena@cicese.mx (J.P.-R.); echenausia@cicese.mx (J.L.E.-M.)

Abstract: In the study of dynamic systems, bifurcation diagrams are a very popular graphical tool for studying stability and nonlinear changes in behavior. They are instrumental in analyzing the system's response to parameter changes. These diagrams show the system's various dynamic patterns and phase transitions by plotting the relationship between the system response and the parameters. This paper presents a computational algorithm for studying bifurcations in dynamic systems, especially for systems with chaotic behavior that depends on parameter changes. Depending on the type of system to be analyzed, the following two strategies for computing bifurcation diagrams are described: (i) detecting crossing points through the Poincaré plane and (ii) the identification of local maxima (or minima) in one of the system solutions. In addition, this paper presents a method for implementing parallel computation in MATLAB using the *Parallel Computing Toolbox* from MathWorks, which significantly reduces the computational time required to generate bifurcation diagrams. This work contributes to the study of dynamic systems by providing the reader with accessible tools for studying any dynamic system under established standards and reducing the computational time required for these types of studies by implementing these algorithms utilizing the multi-core processors found in modern computers.

Keywords: bifurcation diagram; Feigenbaum diagram; dynamical systems; chaos; parallel computing; HPC

MSC: 37C20; 65Y05; 65K05; 65E05

check for updates

Academic Editor: Cristian Lazureanu

Received: 14 April 2025
Revised: 19 May 2025
Accepted: 24 May 2025
Published: 29 May 2025

Citation: Ontañón-García, L.J.; Barajas-Ramírez, J.G.; Campos-Cantón, E.; Magallón-García, D.A.; Guerra-García, C.A.; Cuesta-García, J.R.; Pena-Ramirez, J.; Echenausía-Monroy, J.L. Algorithm for Generating Bifurcation Diagrams Using Poincaré Intersection Plane. *Mathematics* **2025**, *13*, 1818. https://doi.org/10.3390/math13111818

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

For decades, scientists have been fascinated by the intricate interplay of order and chaos in dynamical systems. From the graceful orbits of celestial bodies to the irregular rhythms of heartbeats, dynamical systems permeate our understanding of nature.

Understanding chaos requires powerful analytical tools that can navigate the complexities of dynamic behavior. Small changes in initial conditions or system parameters—especially in systems exhibiting chaotic behavior—can significantly alter the system's structure. Various methods, such as phase diagrams, Poincaré sections, and power spectrum analysis, can be

employed to analyze these changes. Additionally, exploring the dynamics of a system across a range of parameters provides a broader understanding of its behavior. Bifurcation theory plays a crucial role in this exploration, as it focuses on transition models and helps identify instabilities within systems when parameters are varied [1,2].

A bifurcation represents a qualitative change in the solutions of differential equations as parameters are altered. Bifurcation diagrams graphically depict how a dynamic system's behavior evolves with varying parameter values, illustrating transitions between stable states, limit cycles, and chaotic behavior. These diagrams apply to discrete and continuous systems, playing a crucial role in analyzing the dynamics of various differential equations and iterative maps [3,4]. Notably, the Feigenbaum diagram illustrates the sequence of period-doubling bifurcations as a path to chaos in discrete one-dimensional maps, such as the logistic or Hénon maps, revealing a universal proportion characterized by the Feigenbaum constant, which quantifies the diminishing intervals between successive bifurcations leading to chaos. While the bifurcation diagram is a general concept applicable to various systems, the Feigenbaum diagram focuses on discrete systems exhibiting this self-similar structure.

To construct bifurcation diagrams, the following two standard methods are employed: (i) analytical determination of critical change points in the system and (ii) numerical simulations using integration methods [5–7]. These approaches unveil intricate transition points, known as bifurcations, where ordered trajectories split into various behaviors. Such diagrams are widely applied across scientific fields, often generated through simulations to identify local maxima in time series for each studied bifurcation parameter. For instance, they analyze subharmonic resonance in rotor–ball bearing systems [8], evaluate ecological models of species interactions [9], and study the impact of immunotherapy on tumor growth [10]. Bifurcation diagrams are especially useful for exploring how coupling strength influences transitions between synchronized chaotic and unsynchronized periodic states in coupled systems. Research [11] has mapped system behavior across varying coupling strengths, identifying critical thresholds such as bistability onset and shifts between synchronized and unsynchronized states. These diagrams offer insights into the effects of coupling structures on dynamics and the relationship between feedback mechanisms and initial conditions.

The bifurcation diagrams used in the work described above are based on examining time series, identifying changes in the slope of one of the state variables, and viewing these inflection points as quantitative information about the system's state. While this is one way to create a bifurcation diagram, there is no criterion for selecting the correct state variables to visualize the oscillator's state transitions.

For example, the algorithms proposed in the recent MATLAB R2024b file sharing contributions [12–14] focus on the analysis of local maxima or intersection planes fixed in space. As mentioned in the descriptions of these algorithms, the placement of the intersection planes is influenced by the dynamics of the analyzed system.

In addition to these simulation-based methods, there are established bifurcation software tools such as AUTO [15], MatCont [16], and XPPAUT [17] that use numerical continuation methods to track equilibrium branches, periodic orbits, and bifurcation points. These packages usually require explicit model formulations and Jacobian matrices and are particularly suitable for analytically defined systems. In contrast, the algorithm proposed in this work is based on the detection of crossing events in the Poincaré plane directly from the simulated trajectories, making it applicable to both analytical and black-box models. Moreover, our method allows for the adaptive selection of the orientation of the Poincaré section—a feature not considered by standard tools or recent algorithmic contributions—and benefits from parallel computation to reduce processing time without compromising accuracy.

In this sense, the Poincaré plane is a technique that allows for the visualization of higher-dimensional flows in lower-dimensional spaces [18]. Taking a cross-section of a trajectory at regular intervals helps identify periodic trajectories, quasi-periodic behavior, and chaotic dynamics, becoming essential for understanding non-integrable Hamiltonian systems, stability theory, chaotic attractors, and as an auxiliary tool to calculate the fractal dimension of chaotic systems [19]. Henri Poincaré developed this concept during his work on the three-body problem, particularly in his prize-winning paper for the competition to celebrate the 60th birthday of King Oscar II of Sweden in 1889. With this work, he laid the foundations for what would later become modern chaos theory [20].

The computational resources needed to create these diagrams depend on various factors, including the accuracy of graphics visualizing transitions between chaotic and periodic points, the number of iterations required by the algorithms, and the arrangement of sections and intersection planes with different slopes.

We propose an algorithm that allows the user to study bifurcation dynamics by selecting the orientation of the Poincaré plane based on the system properties to be analyzed. Since computational efficiency is crucial in the analysis of complex dynamical systems, especially in the construction of bifurcation diagrams and the approximation of solutions to differential equations, our approach highlights the importance of optimizing both algorithm design and simulation performance through sequential and parallel programming or high-performance computing [21].

The main contributions of this work can be summarized as follows:

- Poincaré Plane Adaptation: The user-selected slope of the Poincaré plane is adapted to the system's geometry, improving the accuracy of event detection;
- Transient Filtering: A mechanism to efficiently discard transient states and manage output storage, improving the robustness of system response to initial conditions;
- Modular Algorithm Design: Separation of the simulation, detection, and analysis phases, allowing flexible adaptation to different types of systems;
- Parallel Implementation: Use of the MATLAB construct `parfor` to significantly reduce processing time while maintaining numerical accuracy;
- Validation Through Case Studies: Application to well-known systems, including the Rössler and bidirectional Lorenz models, to demonstrate the effectiveness of the algorithm in solving complex bifurcation structures.

The sections of this article are as follows: The mathematical definition and the methodology for constructing a Poincaré plane, as well as the definition of a bifurcation diagram, are discussed in Sections 2 and 3. The numerical results for two chaotic systems used as test oscillators (the Rössler system and a multiscroll generator) are presented in Section 4. Section 5 describes the algorithms and routines used to obtain the results shown. The results of the algorithms used are presented in Section 6, and the conclusions of the work are described at the end of the paper.

2. Mathematical Preliminaries

Following the ideas presented in [22,23], in this section, we present the formal mathematical description of trajectories within a continuous dynamical system, the concept of Poincaré planes, and bifurcation diagrams.

2.1. Poincaré Planes

The use of Poincaré sections to analyze periodic or chaotic behavior is well established in the study of nonlinear dynamical systems [24–26]. To construct a Poincaré plane, we should first consider a continuous dynamical system, expressed as follows:

$$\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}); \tag{1}$$

where the state vector $\mathbf{x} = (x_1, \ldots, x_m)^T \in \mathbb{R}^m$ and the corresponding smooth vector field is denoted by $\mathbf{F} : \mathbb{R}^m \to \mathbb{R}^m$. The system (1) induces a flow $\left(\varphi^t\right)_{t\in\mathbb{R}}$ in the dimensional space $\mathbb{R}^m$ [27]. For each initial condition $\mathbf{x}_0 = \mathbf{x}(0)$, a trajectory $\{\mathbf{x}(t) = \varphi(t, \mathbf{x}_0) : t \geq 0\}$ is associated.

It is important to note that the system possesses a dissipation ball $\mathcal{D} \subset \mathbb{R}^m$, such that $\forall d \in \mathcal{D} : \varphi(t, d) \subset \mathcal{D},\ t \geq 0$. The maximal attractor $\mathcal{A}$ is the largest invariant subset of $\mathcal{D}$ that exhibits attraction. Also, the trajectory in the attractor is expressed as $\mathbf{x}(t) := \varphi_{\mathcal{A}}(t, \mathbf{x}_0)$.

The Poincaré plane is an $(m-1)$-dimensional surface, defined as follows:

$$\Sigma := \{\mathbf{x} \in \mathbb{R}^m : g(\mathbf{x}) = 0\}; \tag{2}$$

where $g(\mathbf{x}) = \alpha_1 x_1 + \ldots + \alpha_m x_m + \alpha_{m+1}$, with $\alpha_1, \ldots, \alpha_{m+1} \in \mathbb{R}$ as coefficients of the hyperplane equation. To ensure that Σ is perpendicular to the flow $\left(\varphi^t\right)_{t\in\mathbb{R}}$, the coefficients and slope are chosen to satisfy the transversality condition $\nabla g(\mathbf{x}) \cdot \mathbf{F}(\mathbf{x}) > 0, \forall \mathbf{x} \in \Sigma$, and to ensure that the trajectory crosses recursively the Poincaré plane at the time instants t_k, $t_k \in \mathbb{R}$ with $k \in \mathbb{N}$. Generating crossing points $\mathbf{p}_k = \varphi_{\mathcal{A}}(t_k, \mathbf{x}_0) \in \Sigma$, where $t_k \in \mathbb{R}$ with $k \in \mathbb{N}$.

To achieve this, we focus on a set of points generated by the trajectory that crosses the Poincaré plane, defined as follows:

$$\mathcal{P} := \{\mathbf{p}_k\}, k = 1, 2, \ldots; \tag{3}$$

with $\mathbf{p}_k = (x_1(t_k), \ldots, x_m(t_k))^T$, where t_k satisfies the following:

$$\alpha_1 x_1(t_k) + \ldots + \alpha_m x_m(t_k) + \alpha_{m+1} = 0. \tag{4}$$

Therefore, $\#\mathcal{P}$ grows as $t \to \infty$, where # denotes the cardinality of the set.

We will then explore the case of three-dimensional benchmark systems, i.e., their solution belongs in $\mathbb{R}^3$. Therefore, the Poincaré plane will be as follows:

$$\Sigma := \left\{(x_1, x_2, x_3)^T \in \mathbb{R}^3 : \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \alpha_4 = 0\right\}. \tag{5}$$

where the values of α_i with $i = 1, \ldots, 4$ will be set according to the slope needed and considering the characteristics of each system analyzed.

2.2. Bifurcation Diagrams

Now, to analyze transitions to chaos, we extend the autonomous system (1) by considering a variational parameter $\mu \in \mathbb{R}$, as follows:

$$\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}; \mu), \tag{6}$$

where $\mathbf{F} : \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}^m$ governs the system's dynamics. The parameter μ modulates qualitative changes in the attractor $\mathcal{A}(\mu)$, which may evolve towards stable equilibria, periodic orbits, or chaotic attractors.

Since a bifurcation diagram visualizes the asymptotic behavior of trajectories $\mathbf{x}(t, \mu) := \varphi_{\mathcal{A}}(t, \mathbf{x}_0, \mu)$ as μ varies [24], we will use the Poincaré map $\mathcal{P}$ from (3) to project intersections with Σ onto a representative coordinate (e.g., x_1, x_2, x_3).

Then, one possible way to define the bifurcation diagram is as follows: Let μ be the bifurcation parameter that takes M equally spaced values from $\mu_{\min}$ to $\mu_{\max}$, with step size

$\Delta\mu$, i.e., $\mu_i = \mu_{\min} + \Delta\mu(i-1)$, for $i = 1, \ldots, M$ and $\Delta\mu = (\mu_{\max} - \mu_{\min})/(M-1)$. Then, the bifurcation diagram $\mathcal{B}$ can be represented by the following geometric locus:

$$\mathcal{B} = \left\{ (\mu_i, x_h^j) \in \mathbb{R}^2 \;\middle|\; x_h^j \in \mathcal{P}(\mu_i, \mathbf{x}_0),\ h \in \{1, \ldots, m\}, j = 1, \ldots, k^i,\ i = 1, \ldots, M \right\},$$

where, with a slight abuse of notation, $\mathcal{P}(\mu_i, \mathbf{x}_0)$ is the matrix version of the set defined in (3), that is, it is a matrix where column j corresponds to $\mathbf{p}_j$, and row h corresponds to the h-th component of the state vector $\mathbf{x}$, i.e., x_h. Note that the number of columns is not necessarily the same for all values of the parameter μ, which is why the superscript in k^i is used as a label. Usually, the crossings obtained from transient states are discarded by considering $j \gg 1$.

3. Numerical Calculation of Poincaré Plane Crossing Events

We use the fourth-order Runge-Kutta (RK4) integration method to numerically solve the system of equations. Since numerical integration produces discrete trajectories rather than continuous ones, the exact location of a crossing event must be estimated. This section outlines the following three approaches for computing bifurcation diagrams from these discrete trajectories: two based on Poincaré crossings and one based on local maxima or minima.

3.1. Detecting a Poincaré Crossing

In the mathematical and analytical context, the crossing events $\mathbf{p}_k$ with the Poincaré plane occur precisely at the instant the event is triggered, due to the continuous nature of the system's trajectory. Given that the systems' numerical solution corresponds to a discrete vector instead of a continuous one, the following assumptions must be considered.

Achieving a situation in which the numerical solution matches the crossing point of the plane at a specific trajectory point is challenging and typically requires an exceedingly small integration step, along with an infinite number of points in the solution. Thus, the crossing event must be calculated as illustrated in Figure 1, using a selection of points that marginally cross the plane in $\mathbb{R}^3$.

In Figure 1a, the thin gray line represents the system's trajectory in continuous time, the semi-transparent surface in light blue represents the Poincaré plane Σ. The graph depicts three points in red, with the first on the left side of the image, called $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, x_3^{(i)})^T$, representing the point of the system's flow's discrete numerical solution. Note that this point is located before the trajectory crosses the plane. Subsequently, on the right side of the plane, the point $\mathbf{x}^{(i+1)} = (x_1^{(i+1)}, x_2^{(i+1)}, x_3^{(i+1)})^T$ is also marked in red, which represents the next point in the discrete solution of the system's flow, that is, given the integration step, there is no solution point between $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(i+1)}$. Now, the red point located on the plane, represented as $\mathbf{p}_k$, represents the crossing point of the continuous trajectory. This point needs to be estimated due to the discrete solution computed by RK4, indicating a crossing point between $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(i+1)}$ for the reasons explained above, so it is necessary to calculate it using the following approximation.

To find the intersection point $\mathbf{p}_k$, we consider the vector $\overrightarrow{\mathbf{x}^{(i)}}$ that starts at the origin and heads toward $\mathbf{x}^{(i)}$ and the vector $\overrightarrow{(\mathbf{x}^{(i)}\mathbf{x}^{(i+1)})}$ that starts at $\mathbf{x}^{(i)}$ and heads toward $\mathbf{x}^{(i+1)}$, both of which are marked with the black arrow lines. Note that the intersection $\mathbf{p}_k$ determines a vector $\overrightarrow{\mathbf{p}_k}$ in the same direction as $\vec{\mathbf{x}}^{(i+1)}$ but of a smaller proportion. Given that $\mathbf{p}_k \in \Sigma$ and through the discussion presented after Equation (3), we can define the following:

$$\overrightarrow{\mathbf{p}_k} = \overrightarrow{\mathbf{x}^{(i)}} + r\overrightarrow{(\mathbf{x}^{(i)}\mathbf{x}^{(i+1)})}; \tag{7}$$

with $\overrightarrow{(\mathbf{x}^{(i)}\mathbf{x}^{(i+1)})} = \langle \mathbf{x}^{(i+1)} - \mathbf{x}^{(i)} \rangle$ and $r \in \mathbb{R}$ that correspond to the proportion of the vector. Thus, an approximation and alternative form to describe Equation (4) with the discrete values obtained in the RK4 integration will be as follows:

$$\begin{aligned} &\alpha_1(x_1^{(i)} + r(x_1^{(i+1)} - x_1^{(i)})) + \ldots \\ &\alpha_2(x_2^{(i)} + r(x_2^{(i+1)} - x_2^{(i)})) + \ldots \\ &\alpha_3(x_3^{(i)} + r(x_3^{(i+1)} - x_3^{(i)})) + \alpha_4 = 0; \end{aligned} \tag{8}$$

Solving Equation (8) for r will result in the following:

$$r = -\frac{\alpha_1 x_1^{(i)} + \alpha_2 x_2^{(i)} + \alpha_3 x_3^{(i)} + \alpha_4}{\alpha_1(x_1^{(i+1)} - x_1^{(i)}) + \alpha_2(x_2^{(i+1)} - x_2^{(i)}) + \alpha_3(x_3^{(i+1)} - x_3^{(i)})}. \tag{9}$$

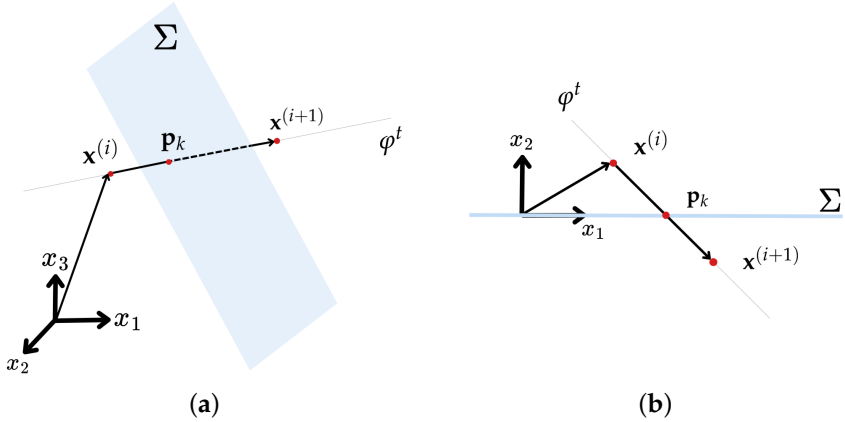

Figure 1. Illustration of Poincaré Plane and crossing point $\mathbf{p}_k$ in, (**a**) $\mathbb{R}^3$, (**b**) $\mathbb{R}^2$, considering numerical solutions of dynamical system.

Crossing in the $x_2 = 0$ Poincaré Plane

Many benchmark dynamical systems exhibiting strange attractors $\mathcal{A}$ and parameter dependence typically oscillate around the origin. As a specific example, this algorithm will examine crossing events $\mathbf{p}_k$ that occur in the plane Σ described in Equation (5), with parameters set as $\alpha_{1,3,4} = 0$ and $\alpha_2 = 1$, positioning them within the $x_2 = 0$ plane. This analysis will replicate the calculations of the crossing events as established in the previous section. A projection onto the R^2 plane is presented in Figure 1b. In this case, the procedure to find the crossing event $\mathbf{p}_k$ due to the numerical discrete points $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(i+1)}$ will be the same as previously discussed, by the calculation of r given in Equation (9).

3.2. Map of the Local Maxima

Bifurcation diagrams can also be constructed using the local maxima (or minima) of the system's flow based on a particular initial condition rather than relying on crossings through the Poincaré plane; in this sense, the transversality condition is no longer needed. This method facilitates the detection of bifurcations in trajectories that are not strictly constrained to a lower domain surface. To illustrate this approach, consider the sequence of point solutions $\mathbf{x}^{(i-1)} = (x_1^{(i-1)}, x_2^{(i-1)}, x_3^{(i-1)})^T$, $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, x_3^{(i)})^T$, and $\mathbf{x}^{(i+1)} = (x_1^{(i+1)}, x_2^{(i+1)}, x_3^{(i+1)})^T$. A set of local maxima $max(\mathbf{x}^{(i)})$ [28], can be identified as follows:

$$\Delta_\eta^{\max} = \left\{ max_\eta(\mathbf{x}^{(i)}) \mid x_\eta^{(i-1)} < x_\eta^{(i)} > x_\eta^{(i+1)} \right\}; \tag{10}$$

where $\eta = 1, 2, 3$ depends on the state where the analysis will be conducted. In essence, we aim to pinpoint specific locations in space where the slope of the trajectory shifts from positive to negative. A similar procedure can be applied to identify local minima

$min_\eta(\mathbf{x}^{(i)})$ in Equation (10), using the criterion $x_\eta^{(i-1)} > x_\eta^{(i)} < x_\eta^{(i+1)}$, where the slope of the trajectory transitions from negative to positive values. The coordinates of the points fulfilling the condition outlined in Equation (10) will be recorded as indicated in the map from Equation (3) for subsequent plotting.

4. Benchmark Systems

To study the algorithm, we will examine continuous dynamical systems in $\mathbb{R}^3$ that exhibit chaotic behavior, gradual changes in the shape of the attractor, and period-doubling cascades that reflect various stable behaviors. For simplicity and without loss of generality in our notation, we will represent the states of these systems using the vector $(x_1, x_2, x_3)^T = (x, y, z)^T$.

4.1. Rössler System

The first example of the bifurcation analysis of the algorithm will be the Rössler system [29], because it presents more continuous, gradual changes in attractor shape and rich parameter variation dynamics. The corresponding equations are described as follows:

$$\begin{aligned} \dot{x} &= -(y+z), \\ \dot{y} &= x + ay, \\ \dot{z} &= b + z(x-b), \end{aligned} \tag{11}$$

where $x, y, z \in \mathbb{R}$ are the states of the system and $a, b, c \in \mathbb{R}$ are the parameters to be analyzed. The system given in Equation (11) results in chaotic trajectories for $a = b = 0.2$ and $c = 5.7$. Thus, we will study the bifurcation diagram for a range of values of the b parameter, maintaining fixed a and c for the different approaches described in Section 2. For illustrating purposes on the system's behavior, Figure 2a depicts the chaotic attractor in $\mathbb{R}^3$. In contrast, Figure 2b shows a projection onto the (x, y) plane. Considering the shape of the Rössler attractor and following the idea presented in [22], the slope of the plane Σ Equation (5) was determined using the following coefficients $\alpha_1 = 0.5934$; $\alpha_2 = -1.1636$; $\alpha_3 = 0$; and $\alpha_4 = -2.4068$.

4.2. Bidirectional Coupled Lorenz Systems

While the algorithm is well suited for the Rössler system, calculating its bifurcation diagram may not be as computationally demanding. Therefore, we include a more complex example of the algorithm here. Specifically, we will consider a pair of bidirectionally coupled Lorenz systems, as presented in [11], whose interconnected dynamics require more detailed analysis. The system is represented by the following equations:

$$\begin{aligned} \dot{\chi}_1 &= f(\chi_1) + k_1(\chi_2 - \chi_1), \\ \dot{\chi}_2 &= f(\chi_2) + k_2(\chi_1 - \chi_2). \end{aligned} \tag{12}$$

In this context, $\chi_{1,2} \in \mathbb{R}^3$ represents the state vector, and $f : \mathbb{R}^3 \to \mathbb{R}^3$ is the vector field that describes the dynamics of an isolated node. The dynamics are governed by the following equations:

$$\chi_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}, \quad f(\chi_i) = \begin{bmatrix} \sigma(y_i - x_i) \\ \rho x_i - y_i - \alpha x_i z_i \\ \epsilon x_i y_i - \beta z_i \end{bmatrix}. \tag{13}$$

where $i = 1, 2$ determines the system, $x_i, y_i, z_i \in \mathbb{R}$ are the state variables, $\sigma, \rho, \beta \in \mathbb{R}_+$ are the system parameters, and $\epsilon \in \mathbb{R}_+$ is a scaling factor. We will utilize the common

parameter values associated with chaotic behavior: $\sigma = 10,\ \rho = 28,\ \beta = \frac{8}{3}$, and we will set a scaling factor of $\epsilon = 10$.

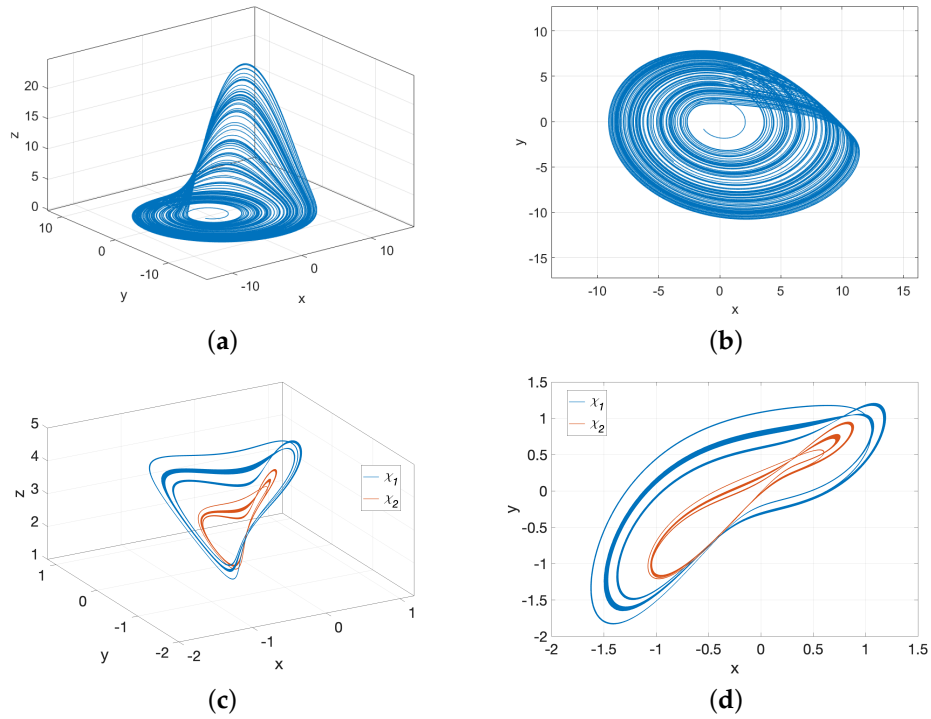

Figure 2. Projection onto phase space for benchmark systems: (**a**) Rössler system given in Equation (11). (**c**) Bidirectional Lorenz system given in Equation (12), with $k_1 = -3.5975, k_2 = 5$. Meanwhile, in (**b**,**d**), attractors are projected into (x, y) plane.

The constants k_1 and k_2 in Equation (12) represent the coupling strengths between the systems. As outlined in [11], we will assume that k_1 is a negative constant ($k_1 \in \mathbb{R}_-$), while k_2 is a positive constant ($k_2 \in \mathbb{R}_+$).

The specific values of these coupling strengths lead to rich dynamic behaviors, including periodic, stable, and chaotic states. It is also crucial to note that the systems belong to $\mathbb{R}^3$. However, since the interconnection between them needs to be simulated simultaneously for both systems, we will be integrating them as a single system in $\mathbb{R}^6$. Figure 2c depicts the attractors χ_1 (in blue line) and χ_2(in orange line) in $\mathbb{R}^3$ for a coupling force of $k_1 = -3.5975$ and $k_2 = 5$. And Figure 2d shows a projection onto the (x, y) plane.

5. Algorithms for Computing Bifurcation Diagrams

5.1. Architecture Specifications

The algorithms were implemented using MATLAB R2024b Update 4 (24.2.0.2833386) on a 64-bit (maca64) system. We will explore two approaches. Both are available for download as part of the Supplementary Materials. The first approach is a sequential algorithm that operates using iterative `for-loops` with the regular MATLAB worker assigned to the computer's processor.

In this case, the algorithm's processing time heavily depends on the number of divisions applied to the bifurcation parameter being studied and the length of the time vector, i.e., the number of time steps in the solution.

The second algorithm approach considers `parfor-loops` iterations distributed among parallel workers in a parallel pool. This approach is based on the Parallel Computing Toolbox of MathWorks, and the processing time also depends on the number of divisions implemented in the bifurcation parameter to be studied, the length of the simulated time,

and the time step. However, as discussed later, it strongly depends on the number of cores of the running computer, as well as the number of selected workers by the parallel distribution.

5.2. Computation of a Sequential Bifurcation Diagram Using Iterative For-Loops

The initial method for computing bifurcation diagrams utilizes standard iterative for-loops without parallel distribution. This approach does not depend on toolboxes outside the base MATLAB platform. You can confirm this by using the command `[fList, pList] = matlab.codetools.requiredFilesAndProducts('run_main.m')`, which will show the dependencies of the sequential algorithm. As a result, the output in `pList.Name` will reflect only the components included in the MATLAB distribution.

Sequential Algorithm Overview

The algorithm systematically varies the bifurcation parameter and numerically solves the system for each value using the Runge–Kutta 4th order method (RK4). Typically, the time required to obtain the solution trajectories is longer than the processing time needed to detect the bifurcation points; however, this largely depends on the type of numerical integrator used. In the case of RK4, the algorithm implements four approximations for each solution point before evaluating the next point along the trajectory.

Once the system's solution is obtained, we consider discarding at least 50–60% of the initial transient states due to the system's complexity.

As a result, the bifurcation points are extracted using the methods mentioned earlier—Poincaré plane intersection, zero-crossing detection, and local maxima computation. The results are then stored and visualized in the final stage. Listing 1 presents the pseudo-code for this MATLAB implementation.

Listing 1. Pseudo-code for iterative `for-loop` algorithm computing bifurcation diagram.

```
1.  Initialize the~workspace.

2.  Define:
    - System parameters
    - Initial conditions
    - Bifurcation parameter~range

3.  Preallocate memory for~results.

4.  For~each value of the bifurcation parameter:
    a. Integrate the system using Runge-Kutta 4th order.
    b. Remove transient data from the time series.
    c. Compute bifurcation points using:
        - Zero-crossing detection
        - Poincare plane crossings
        - Local maxima extraction
    d. Store bifurcation~data.

5.  Plot and save the bifurcation~diagram.

6.  End algorithm.
```

The main program is divided into five sub-files. As mentioned previously, the processing time depends on the number of divisions implemented in the bifurcation parameter to be studied, the length of the simulated time, and the time step. The following is a list describing the main file and each of the sub-files included:

- run_main.m: This is the main file for the algorithm. Users must select a bifurcation parameter within a specified range of values, ensuring that these values produce

bounded trajectories in the integration method. The algorithm is limited to a fixed number of iterations, and a percentage of these iterations is later discarded as transient states, with the specific number determined by the user. The algorithm applies a designated function or subroutine for each integration method and bifurcation test. It is important to note that the algorithm does not exclude solutions even if the trajectories continue to grow over time. This can lead to inaccurate bifurcation diagrams with larger dimensions. Additionally, users can choose from various bifurcation detection algorithms (described in the subsequent sections) to analyze the system's trajectories. If the process requires a substantial number of iterations to find a solution, the resulting data can be saved as a *.mat file to help manage the processing time of the algorithms.

- DynamicalSystem.m: The equations for continuous dynamical systems must be included in this file. All bifurcation parameters considered in the study should be defined as global MATLAB variables to facilitate setting their values in the main file. The system should be configured to have one of its parameters vary within a specified range, as seen in the pseudo-code presented in Listing 2. The number of degrees of freedom for the system's algorithm is $\mathbb{R}^3$. However, users may modify this with the size of the initial condition vector and select the proper systems with larger dimensionality. In this particular case, the Rössler system [29] is used as a benchmark example due to its chaotic behavior at certain parameter values.
- RungeKuttaBif.m: This routine implements the solution for dynamical systems using a numerical integration method with a fixed step size. We utilize the Runge–Kutta fourth-order method, which allows users to adjust the step size as needed. However, a step size of 0.01 is generally sufficient for most benchmark systems. While MATLAB's `ode` functions might be suitable depending on the complexity of the equations, we will not consider them in this work.
- Bif_ZeroCrossing.m: This routine determines and plots the bifurcation diagram of the system by analyzing the crossing events $\mathbf{p}_k$ in the plane Σ, as described in Equation (5). In this case, the slope parameters are set to $\alpha_{1,3,4} = 0$ and $\alpha_2 = 1$, as mentioned in Section 2.2. This method is applicable to systems where $\mathcal{A}$ oscillates around the origin. Specifying a bifurcation parameter as part of the input global variables in the main file and a valid range of values is essential. The resolution and quality of the bifurcation diagrams depend on the initial conditions provided to the system.
- Bif_PoincarePlane.m: This routine calculates the bifurcation and plots the bifurcation diagram of the system considering the crossing events $\mathbf{p}_k$ in the plane Σ from Equation (5), with $\alpha_{1,\ldots,4}$ set by the user. This method is applicable to systems that oscillate throughout space, provided that the trajectory intersects the Poincaré plane via the coefficients α_i.
- Bif_LocalMaxima.m: This routine calculates the local maxima of the system's trajectory and generates the map given in Equation (10) with their location coordinates in space. If the dynamics of the system to be analyzed by the parameter variation is not completely known, this method will present a feasible alternative to plot their chaotic or periodic orbits. The program can be easily modified if the user needs the local minimum.

5.3. Parallelized Bifurcation Diagram Computation Using the Parallel Computing Toolbox

To improve computational efficiency, an alternative implementation was considered—distributing the iterations of the loop across multiple workers using MathWorks' Parallel Computing Toolbox. This method allows computations to run simultaneously across several CPU cores, significantly reducing processing time.

Listing 2. Pseudo-code for `DynamicalSystem.m` function (Rössler system definition).

```
Function DynamicalSystem(t, Y)
1.  Initialize output as a zero vector with the same length as~Y.

2.  Assign state variables:
    - x ← Y(1)
    - y ← Y(2)
    - z ← Y(3)

3.  Load global parameters:
    - a, b, c

4.  Define system equations:
    - output(1) ← - (y + z)
    - output(2) ← x + a * y
    - output(3) ← b + z * (x - c)

5.  Return output vector.
End Function
```

Parallel Algorithm Overview

The core algorithm retains its iterative nature, but instead of a standard `for-loop`, a parallel `parfor-loop` distributes computations among the available/selected workers. The number of active workers is determined by the user's configuration settings for the parallel pool, or they can specify the number of workers using the `parpool` function. In addition, for this program, we will use the same dependency facilities as in sequential computation, requiring only the MATLAB and Parallel Toolbox packages, as commented before in Section 5.2.

In this implementation, the algorithm is designed to take advantage of the capabilities of the Parallel Computing Toolbox by assessing the computer's specifications and the number of cores in the processor to generate an optimal number of MATLAB workers. Due to the specifications of `parfor`, we cannot use global parameters in the code at this stage. As a result, the structure of this implementation differs slightly from that of the previous algorithm. Listing 3 presents the pseudo-code corresponding to this MATLAB code.

The main program is divided into three sub-files to calculate the bifurcation diagrams. The processing time depends on the number of divisions implemented in the bifurcation parameter to be studied, the length of the simulated time, and the time step. The following list describes the main file and each of the sub-files included:

- run_main_parfor.m: This is the main file of the algorithm and where the numerical integrations are processed. The user can select the number of parallel workers in this file using the Parallel Toolbox. This is adjusted with the command `parpool(W)`, where W represents a valid number of cores available on the user's machine. In our example, this line will be commented out so that the algorithm uses the maximum number of available cores. A key difference between a `for-loop` and a `parfor-loop` is that the latter cannot use global variables. Therefore, the system and its dimensions must be adjusted by selecting the initial conditions.
- Bif_ZeroCrossingPAR.m: This routine is similar to the sequential algorithm, which calculates the crossing events through $x_2 = 0$ for the selected systems and plots them. However, it distributes the calculations over the bifurcation parameter vector in the parallel pool.

- Bif_PoincarePlanePAR.m: Using a `parfor-loop`, this one calculates the bifurcation diagram of the system considering the crossing events $\mathbf{p}_k$ in the plane Σ from Equation (5), with $\alpha_{1,\ldots,4}$ set by the user.
- Bif_LocalMaximaPAR.m: Similarly, this routine calculates the local maxima of the system's trajectory with a `parfor-loop`.

Listing 3. Pseudo-code for the parallel bifurcation diagram algorithm using `parfor`.

```
1.  Initialize simulation parameters:
    - Select parpool size (optional)
    - Define bifurcation parameter range b
    - Choose the dynamical system and initial~condition

2.  Preallocate arrays for storing state~trajectories.

3.  Integrate the system using the Runge-Kutta 4th order method in~
    parallel.

4.  Remove transient states (e.g., first 50%) to define working
    dataset: Temporal

5.  Compute bifurcation diagram via Zero-Crossing Detection:
    For each trajectory x_i(t), in~parallel:
        a. Detect crossings of x_i(t) through zero with negative
           slope
        b. Interpolate crossing coordinates (X, Y)
        c. Store parameter values b_i and crossing points
        d. Plot bifurcation diagram: b_i vs.~X

6.  Compute bifurcation diagram via Local Maxima:
    For each trajectory x_i(t), in~parallel:
        a. Identify local maxima in x_2(t)
        b. Store values and corresponding b_i
        c. Plot bifurcation diagram: b_i vs. local maxima of~y

7.  Compute bifurcation diagram via Poincare Plane Intersections:
        a. Detect crossings of the trajectory through the defined
           plane
        b. Interpolate and store intersection points (X, Y) and
           parameter b_i
        c. Plot bifurcation diagram: b_i vs.~X

8.  Optionally save computed data for~post-processing.

9.  End algorithm.
```

6. Algorithm Functionality and Results

Given that the execution time of the algorithm used to generate the solution for the dynamical system heavily depends on the precision of the bifurcation parameter vector and the solution's time step, it is generally recommended to run the program in two parts. At the end of the code, there is an option to save the system solutions using the "save" command. This allows the program to be executed in the following two phases: one for generating the numerical solutions of the system and the other for analyzing in-plane crossings and local maxima. Additionally, a visual aid in a progress bar has been implemented to help track progress and estimate execution times. The following flowchart (Figure 3) describes the subsequent process.

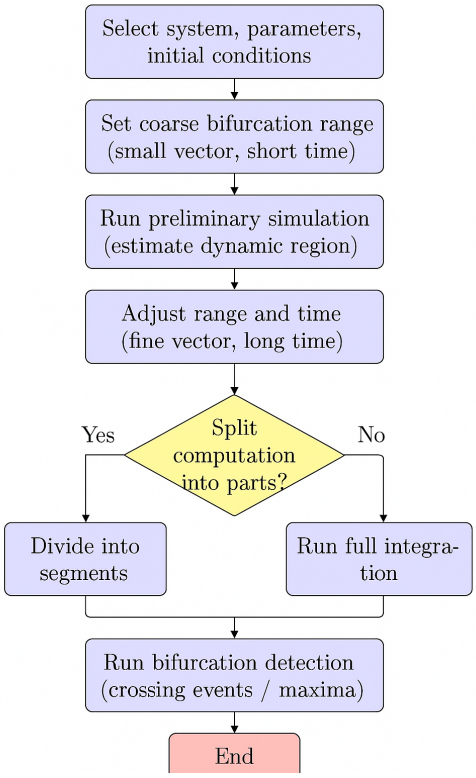

Figure 3. Flowchart of bifurcation algorithm implementation process.

The algorithm's implementation begins with the user selecting the dynamical system of interest and its associated parameters and initial conditions. Due to the event-based nature of the bifurcation detection procedure, a preliminary execution—referred to as a prerun—is recommended, using a reduced bifurcation vector and a limited number of integration steps. This step estimates the approximate location of the event-triggering conditions (e.g., Poincaré crossings) and informs the refinement of the bifurcation parameter range. Once the event locations are approximated, the bifurcation vector is adjusted to achieve the desired resolution. The algorithm is modularized into discrete parts to optimize computational efficiency and ensure robustness against long simulation times. This segmentation enables the storage and recovery of intermediate results, mitigating the risk of data loss during extended runs. Notably, the most computationally intensive component—the numerical integrator—can be executed and stored independently before the full bifurcation analysis. This allows for the reuse of the integrator output, streamlining the subsequent stages of the algorithm.

We now present the plotting of bifurcation diagrams $\mathcal{B}$ resulting from the analysis of the two different systems.

6.1. Bifurcation Diagrams of the Rössler System

First, for the benchmark system example of the Rössler attractor given in Equation (11), a range of b is implemented for $0.1 \leq b \leq 2$ considering 500 steps, as presented in Listing 1. The time solution was calculated for 1×10^5 with a time step of 0.01, removing the first 65% transient states. Figure 4a shows the bifurcation diagram of the crossing events for a Poincaré plane located in the $y = 0$ plane, as stated in Section 2. The values in which the system presents periodic or chaotic orbits can be easily identified here. Take, for instance, the values of $b = 0.105$, $b = 0.5$, $b = 1$, and $b = 1.75$, for which the system presents a 6-period orbit, chaotic behavior, a 2-period orbit, and a 1-period orbit, respectively, as the red dashed lines represent. This can be seen in greater detail in the individual projections in Figure 5. The dark blue line corresponds to the projection in space (a) and onto the

(x, y) plane (b). The Poincaré plane is depicted in a gray mesh or gray line, and the red asterisk represents the crossing events in the selected negative predefined slope (from top to bottom).

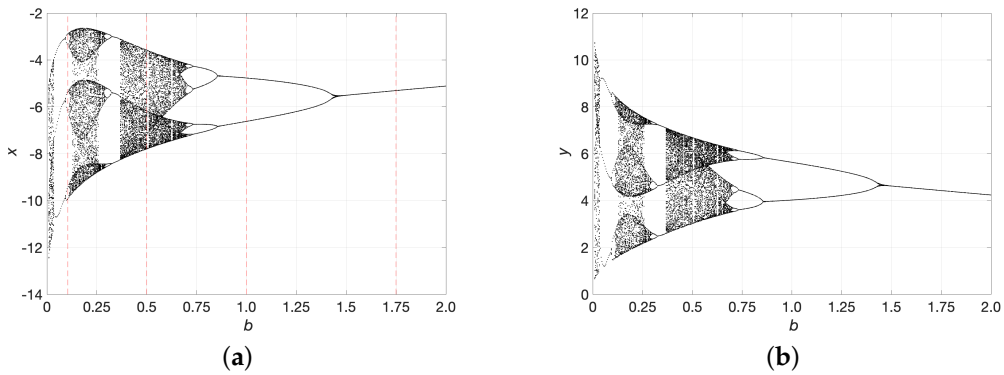

Figure 4. (**a**) Bifurcation with the Poincaré plane; (**b**) Bifurcation with Zero crossing, both with Rössler system given in Equation (11).

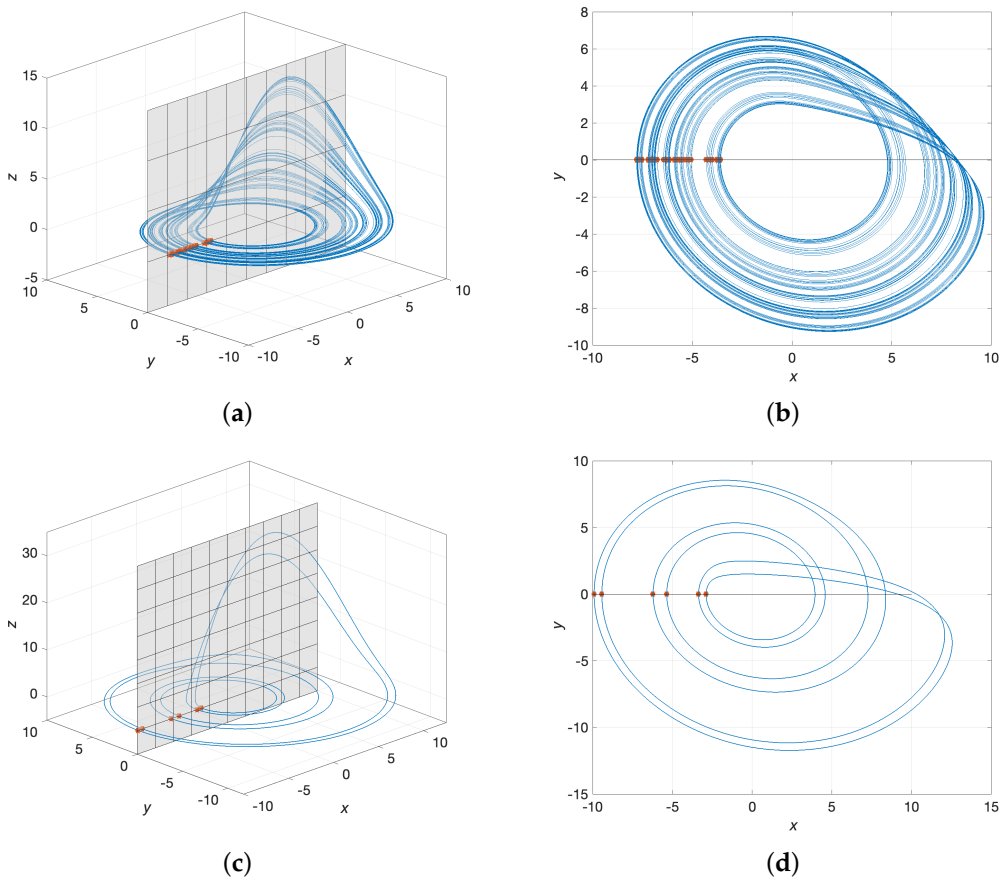

Figure 5. Selection of Rössler system given in Equation (11). (**a**,**b**) $b = 0.5$. (**c**,**d**) $b = 0.105$.

On the other hand, Figure 4b presents the bifurcation diagram obtained by the local maxima. The behaviors resulting from values of $b = 0.105$, $b = 0.5$, $b = 1$, and $b = 1.75$, for which the system presents a 6-period orbit, chaotic behavior, a 2-period orbit, and a 1-period orbit, respectively, are the same as in Figure 4a.

6.2. Bifurcation Diagrams of the Bidirectional Lorenz System

The bifurcation diagram of the systems presented in Equation (12), calculated by the Poincaré plane located at $y = 0$, is presented in Figure 6 (a) for χ_1, and (b) for χ_2. We

considered a variation in the coupling force k_1 parameter between $-4 \leq k_1 \leq -3$ with $k_2 = 5$. The simulation considered a bifurcation vector size of 600 values and 1×10^5 time steps, removing 50% of the transient states. As mentioned in Section 4, the systems were simulated with an RK4 in $\mathbb{R}^6$. However, the bifurcation algorithm remained in $\mathbb{R}^3$. Both diagrams depict the intrinsic behavior of the systems for small variations in the coupling force.

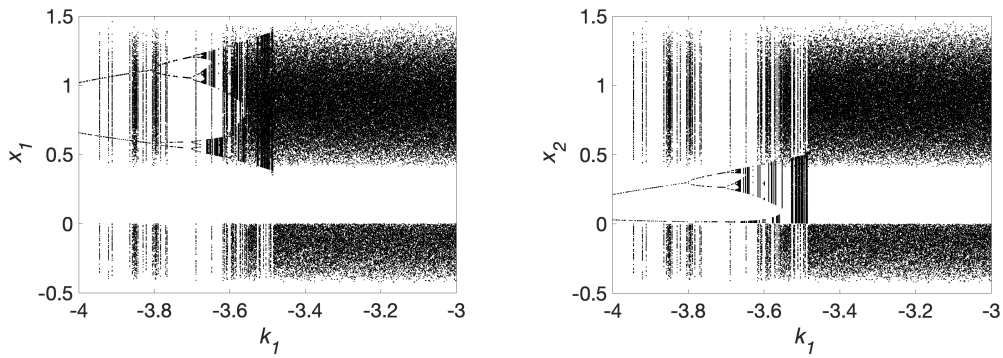

Figure 6. Zero crossing of bidirectional Lorenz systems given in Equation (12) for a range of the coupling force k_1 and $k_2 = 5$ for (**a**) χ_1; (**b**) χ_2.

7. CPU Performance Comparison Between Sequential and Parallel Algorithms

To evaluate the performance of the two versions of the algorithms, we used the `cputime` function in MATLAB, which measures the total CPU time utilized in seconds, following the recommendations provided by MathWorks for MATLAB scripts, specifically utilizing the cputime function to evaluate computational efficiency [30]. We define τ_s and τ_p as the execution time for the sequential and parallel algorithms, respectively. Our performance calculations focus solely on the numerical integration method and evaluate only the zero plane crossing detection, excluding waitbar commands and figure plots.

The solution of the bifurcation diagram is influenced by the system's complexity, the number of time steps considered, and the precision or number of values used for the bifurcation parameter. We explore two general cases of the Rössler system, as described above. First, we use a vector with 1×10^5 time step solution and a bifurcation parameter vector of varying size and precision. Specifically, we construct the vector using the function $\texttt{linspace}(\mu_{min}, \mu_{max}, M)$, where $\mu_{min} = 0.01$ for the Rössler bifurcation parameter b and $\mu_{max} = 2$. The vector size M was calculated from 20 to 300 precision divisions. Figure 7a illustrates the results of this experiment, showing that the sequential algorithm time, τ_s (depicted in blue), increases from 1.04 s for the bifurcation vector with 20 precision data to 16.25 s for a vector of 300 bifurcation values. In contrast, the parallel algorithm time τ_p (represented by the orange line) decreases, ranging from 0.36 s to 2.09 s for the same range of bifurcation vector sizes.

When we increased the time steps of the simulation to 1×10^6 with the same bifurcation parameter vector size M from 20 to 300, we observed an increase in CPU execution times, as shown in Figure 7b. Here, the sequential algorithm time ranged from 18.8 s to 253.14 s, while the parallel case showed a time range of 1.26 s to 15.18 s.

Although the simulation times for the sequential case are not excessive, it is important to highlight that the Rössler system requires relatively less computational power for bifurcation analysis when compared to more complex systems, such as the Lorenz system. Hence, we conducted similar experiments using the same parameters for the bidirectional Lorenz

systems. Figure 7c presents the results for the first case with 1×10^5 solution times and bifurcation parameters ranging from 20 to 300 precision divisions. The sequential algorithm took approximately 45.52 s compared to 3.15 s for the parallel implementation. For the second case with 1×10^6 solution times and the same range of bifurcation parameters, Figure 7d illustrates a significant difference in performance; the sequential algorithm took 3854.34 s while the parallel algorithm completed in only 59.18 s. This shows a considerable reduction in execution time when using the parallel algorithms.

From the graphs shown in Figure 7, we can see that they all have a linear trend depending on the vector size containing the values for the bifurcation parameter; this allows us to determine the algorithm's complexity. Since the computation time in our case only takes into account the numerical integration (the $\mathcal{O}(n)$) and the detection of the zero crossing (the $\mathcal{O}(1)$), it follows that the algorithm presented in this paper has a complexity of $\mathcal{O}(n)$, which is confirmed by the above graphs.

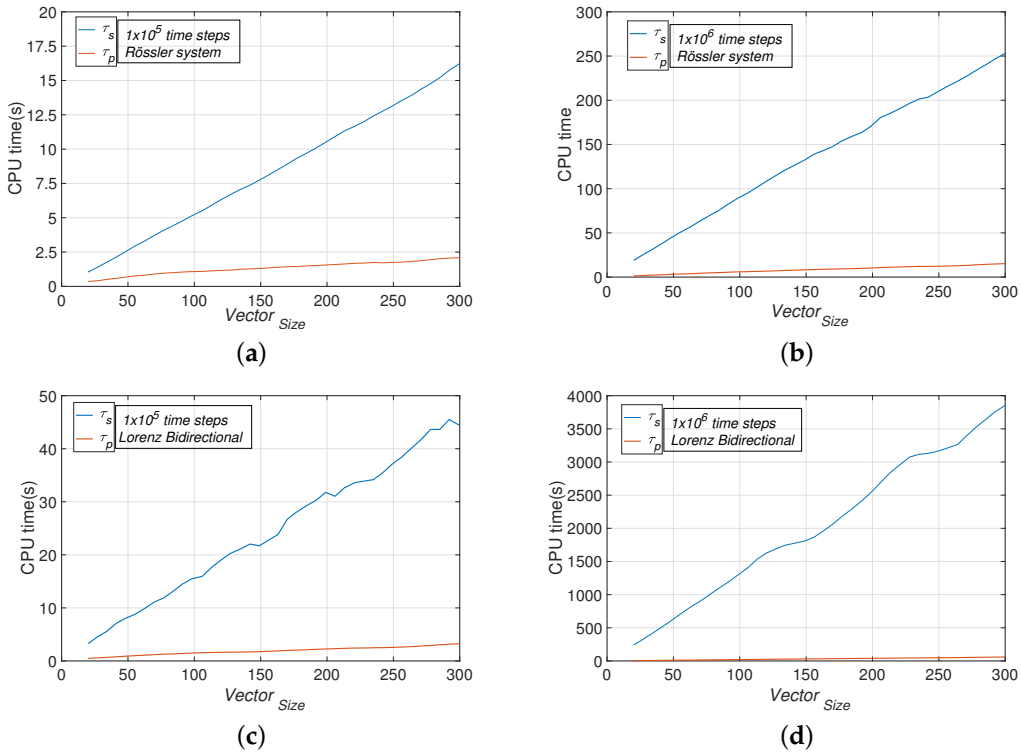

Figure 7. CPU time performance comparison for different time integration steps between sequential and parallel algorithms. For (**a**,**b**) Rössler system given in (11); (**c**,**d**) the bidirectional Lorenz systems given in (12).

Finally, to extend the validation of the proposed algorithms beyond classical chaotic systems, we implemented two well-known nonlinear oscillators in $\mathbb{R}^2$ that exhibit periodic behavior. However, they transition from periodic to chaotic behavior under particular forcing conditions. The first system is the forced Duffing oscillator [31], described by the state-space representation, $\dot{x_1} = x_2; \dot{x_2} = -\alpha x_2 - \gamma x_1 - \delta x_1^3 + k_1 \cos(\omega t)$, where the parameters are set as $\alpha = 0.4$, $\gamma = -0.25$, $\delta = 0.5$, and $\omega = 0.5$. The second system is the forced Van der Pol oscillator [32], expressed as $\dot{x_1} = x_2; \dot{x_2} = 0.1(1 - x_1^2)x_2 - x_1 + k_1 \cos(\omega t)$, with $\omega = 0.5$. These systems were selected for their ability to generate complex periodic responses and transitions toward chaotic regimes when varying the forcing amplitude k_1. Additionally, the Rössler system in $\mathbb{R}^3$ and the bidirectional coupled Lorenz system in $\mathbb{R}^6$ were included to represent more complex and high-dimensional chaotic dynamics.

The computational performance comparison between the sequential and parallel implementations is summarized in Table 1. In this case, a bifurcation vector size of $M = 600$ and $i \times 10^6$ time steps in the simulation and removing 50% of the transient states, as these values resulted in a more suitable and visually appealing bifurcation diagram, as the graphs in Figures 4, 6, and 8 depict. The experiments confirm that, across all systems, the parallel implementation significantly outperforms the sequential version regarding CPU time reduction. Notably, the performance gain is particularly substantial in the bidirectional Lorenz system, where the execution time dropped from approximately 15,739.44 s to 84.62 s, achieving a speedup of 186×. In the lower-dimensional systems, such as the forced Duffing and forced Van der Pol oscillators, the observed speedup is approximately 20×, while for the Rössler system, the gain is around 15×. These results highlight that the parallel algorithm scales efficiently with both the system dimension and the resolution of the bifurcation parameter space, making it suitable for extensive bifurcation studies even in computationally demanding scenarios.

Table 1. Performance comparison of sequential and parallel algorithms across different dynamical systems.

System	Dim	Type	Bifur. Param	CPU Time (Seq.) (s)	CPU Time (Par.) (s)	Speedup
Forced Duffing	2	Periodic-Chaotic	k_1	515.03	25.60	20.11×
Forced Van der Pol	2	Periodic-Chaotic	k_1	521.55	25.72	20.27×
Rössler	3	Chaotic	b	498.99	27.00	15.26×
Lorenz Bidirectional	6	Chaotic	k_1	15,739.44	84.62	186.00×

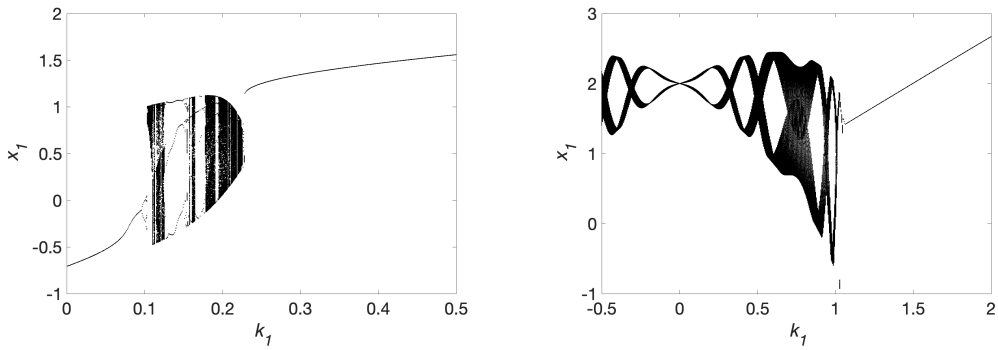

Figure 8. Zero crossing of the (**a**) Force Duffing Oscillator for a range of the coupling force $0 \leq k_1 \leq 0.5$; (**b**) Forced Van der Pol Oscillator for $-0.5 \leq k_1 \leq 2$.

8. Concluding Remarks and Future Work

This paper presents a computational framework for bifurcation analysis in continuous dynamical systems that integrates three algorithmic strategies based on the rigorous formulation of discrete output vectors derived from numerical integration. By using Poincaré section crossings and local maxima, the method accurately identifies critical transitions in the system behavior. The inclusion of transient removal and adaptive storage further improves the detection of asymptotic dynamics, especially for systems with high sensitivity.

The effectiveness of the algorithm was validated using the Rössler and bidirectional Lorenz systems, successfully capturing complex bifurcation phenomena such as period

doubling cascades and transitions between chaotic and periodic regimes. These results emphasize the robustness and adaptability of the system in solving complicated dynamic structures. Proof of this is the use of the proposed tools in the dynamical exploration of the Duffing forced system and the Van der Pol oscillator, two-dimensional dynamical systems (with periodic and chaotic behavior), where our algorithm efficiently computes the corresponding bifurcation diagrams.

A notable contribution is the integration of parallel computing, which significantly reduces computation time (as shown in Table 1). By isolating computationally intensive tasks—namely numerical integration and event detection—parallelization enables efficient scaling in terms of both simulation length and bifurcation resolution.

Despite its strengths, the approach has areas for improvement. The detection of events based on Poincaré crossings and local maxima can be affected by noise, possibly creating false bifurcation points or obscuring true transitions. Furthermore, selection of the appropriate Poincaré sections in higher-dimensional systems remains difficult and often requires adaptive or data-driven strategies to preserve transversality and recursion. The current framework also assumes smooth vector fields and does not directly consider hybrid or discontinuous systems where trajectory changes are abrupt. However, for piecewise systems, such as the one discussed in [33], a suitable choice of the slope of the Poincaré plane allows for an appropriate adaptation of the method, making our tool applicable in such a case.

Looking to the future, the algorithm's applicability can be extended to a number of real-world domains. For example, in neuroscience, transitions between spiking and bursting dynamics can signal pathological states; in epidemiology, bifurcations can reveal tipping points in disease propagation; and in electrical engineering, oscillating circuits often show transitions between stable and unstable regimes.

Future research should address the integration of noise and time delays in system trajectories, benchmarking robustness and adaptation to non-smooth systems. The inclusion of an adaptive bifurcation parameter stepping strategy would enable dynamic refinement near critical points and coarsening in stable regions, thereby improving computational efficiency. Furthermore, the extension of the framework to higher-dimensional models, the implementation of adaptive step size control, and the use of machine learning for the automatic classification of dynamic regimes would further increase the benefits for research in the field of nonlinear dynamics.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/math13111818/s1, Two approaches.

Author Contributions: L.J.O.-G.: Conceptualization, Writing—Original Draft, Writing—Review and Editing, Methodology, Validation, Data Curation, Visualization, Project Administration. D.A.M.-G.: Writing—Review and Editing, Methodology, Validation. C.A.G.-G.: Writing—Review and Editing, Resources, Funding Acquisition. J.G.B.-R.: Writing—Review and Editing, Project Administration, Funding Acquisition. E.C.-C.: Visualization, Methodology, Validation. J.R.C.-G.: Review and Editing, Visualization, Methodology, Validation. J.P.-R.: Writing—Review and Editing, Project Administration, Validation. J.L.E.-M.: Writing—Review and Editing, Resources, Project Administration, Funding Acquisition. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Potosino Council of Science and Technology (COPOCYT) with the Trust project 23871 of the 2023-01.

Data Availability Statement: The data used to support the findings of this study are available on the following website: https://github.com/luisjavier-ontanon/ParallelBifurcationDynamicalSystem (accessed on 19 May 2025).

Acknowledgments: L.J.O.-G. thanks the Potosino Council of Science and Technology (COPOCYT) for the support in Trust project 23871 of the 2023-01 Call. J.L.E.-M. thanks CONAHCYT for their support (CVU: 706850).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Arnold, V.I.; Wassermann, G.; Thomas, R. *Catastrophe Theory*; Springer: Berlin, Germany, 1986; Volume 3.
2. Strogatz, S.H. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering (Studies in Nonlinearity)*; Westview Press: Boulder, CO, USA, 2001; Volume 1.
3. Baker, G.L.; Gollub, J.P. *Chaotic Dynamics: An Introduction*; Cambridge University Press: Cambridge, UK, 1996.
4. Peitgen, H.O.; Jürgens, H.; Saupe, D.; Feigenbaum, M.J. *Chaos and Fractals: New Frontiers of Science*; Springer: New York, NY, USA, 2004; Volume 106.
5. Luque, B.; Lacasa, L.; Ballesteros, F.J.; Robledo, A. Feigenbaum graphs: A complex network perspective of chaos. *PLoS ONE* **2011**, *6*, e22411. [CrossRef] [PubMed]
6. Sprott, J.C. A proposed standard for the publication of new chaotic systems. *Int. J. Bifurc. Chaos* **2011**, *21*, 2391–2394. [CrossRef]
7. Jafari, A.; Hussain, I.; Nazarimehr, F.; Golpayegani, S.M.R.H.; Jafari, S. A simple guide for plotting a proper bifurcation diagram. *Int. J. Bifurc. Chaos* **2021**, *31*, 2150011. [CrossRef]
8. Hou, L.; Chen, Y.; Cao, Q.; Zhang, Z. Turning maneuver caused response in an aircraft rotor-ball bearing system. *Nonlinear Dyn.* **2015**, *79*, 229–240. [CrossRef]
9. McAllister, A.; McCartney, M.; Glass, D.H. Food, fertilizer and Feigenbaum diagrams. *Int. J. Math. Educ. Sci. Technol.* **2024**, *55*, 171–183. [CrossRef]
10. Sourailidis, D.; Volos, C.; Moysis, L.; Meletlidou, E.; Stouboulos, I. The study of square periodic perturbations as an immunotherapy process on a tumor growth chaotic model. *Dynamics* **2022**, *2*, 161–174. [CrossRef]
11. Ontañón-García, L.J.; Cantón, I.C.; Ramirez, J.P. Dynamic behavior in a pair of Lorenz systems interacting via positive-negative coupling. *Chaos Solitons Fractals* **2021**, *145*, 110808. [CrossRef]
12. Moysis, L. Bifurcation Diagram for the Lorenz Chaotic System. MATLAB Central File Exchange. 2025. Available online: https://www.mathworks.com/matlabcentral/fileexchange/156752-bifurcation-diagram-for-the-lorenz-chaotic-system (accessed on 6 April 2025)
13. Moysis, L. Density-Colored Bifurcation Diagrams. MATLAB Central File Exchange. 2025. Available online: https://www.mathworks.com/matlabcentral/fileexchange/156757-density-colored-bifurcation-diagrams (accessed on 6 April 2025).
14. Sylvestre-Baron, P. Plotting Bifurcation Diagram. MATLAB Central File Exchange. 2025. Available online: https://www.mathworks.com/matlabcentral/fileexchange/72057-plotting-bifurcation-diagram (accessed on 6 April 2025).
15. Doedel, E.; Oldeman, B. AUTO-07P: Continuation and Bifurcation Software for Ordinary Differential Equations. Concordia University. 2007. Available online: http://indy.cs.concordia.ca/auto/ (accessed on 6 April 2025).
16. Dhooge, A.; Govaerts, W.; Kuznetsov, Y.A. MATCONT: A MATLAB package for numerical bifurcation analysis of ODEs. *ACM Trans. Math. Softw. (TOMS)* **2003**, *29*, 141–164. [CrossRef]
17. Ermentrout, B. *Simulating, Analyzing, and Animating Dynamical Systems: A Guide to XPPAUT for Researchers and Students*; SIAM: Philadelphia, PA, USA, 2002.
18. Poincaré, H. Sur le problème des trois corps et les équations de la dynamique. *Acta Math.* **1890**, *13*, A3–A270.
19. Echenausía-Monroy, J.L.; Quezada-Tellez, L.A.; Gilardi-Velázquez, H.E.; Ruíz-Martínez, O.F.; Heras-Sánchez, M.d.C.; Lozano-Rizk, J.E.; Cuesta-García, J.R.; Márquez-Martínez, L.A.; Rivera-Rodríguez, R.; Pena Ramirez, J.; et al. Beyond Chaos in Fractional-Order Systems: Keen Insight in the Dynamic Effects. *Fractal Fract.* **2024**, *9*, 22. [CrossRef]
20. Poincaré, H. *Les Méthodes Nouvelles de la Mécanique Céleste*; Gauthier-Villars et fils, Imprimeurs-Libraires: Paris, France, 1893; Volume 2.
21. Garland, M.; Le Grand, S.; Nickolls, J.; Anderson, J.; Hardwick, J.; Morton, S.; Phillips, E.; Zhang, Y.; Volkov, V. Parallel computing experiences with CUDA. *IEEE Micro* **2008**, *28*, 13–27. [CrossRef]
22. Ontanón-Garcia, L.J.; Campos-Cantón, E.; Femat, R.; Campos-Cantón, I.; Bonilla-Marín, M. Multivalued synchronization by Poincaré coupling. *Commun. Nonlinear Sci. Numer. Simul.* **2013**, *18*, 2761–2768. [CrossRef]
23. Ontanón-Garcia, L.J.; Campos-Cantón, E. Discrete coupling and synchronization in the insulin release in the mathematical model of the β cells. *Discret. Dyn. Nat. Soc.* **2013**, *1*, 427050.
24. Guckenheimer, J.; Holmes, P. *Nonlinear Oscillations, Dynamical Systems, and Bifurcations of Vector Fields*; Springer: New York, NY, USA, 1983.
25. Strogatz, S.H. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*; CRC Press: New York, NY, USA, 2018.

26. Ott, E. *Chaos in Dynamical Systems*; Cambridge University Press: Cambridge, UK, 2002.
27. Perko, L. *Differential Equations and Dynamical Systems*; Springer Science & Business Media: New York, NY, USA, 2013; Volume 7.
28. Hancock, H. *Theory of Maxima and Minima*; Ginn: New York, NY, USA, 1917.
29. Rössler, O.E. An equation for continuous chaos. *Phys. Lett. A* **1976**, *57*, 397–398. [CrossRef]
30. MathWorks. Measure Performance of Your Program. 2024. Available online: https://www.mathworks.com/help/matlab/matlab_prog/measure-performance-of-your-program.html (accessed on 6 April 2025).
31. Barba-Franco, J.; Gallegos, A.; Jaimes-Reátegui, R.; Muñoz-Maciel, J.; Pisarchik, A. Dynamics of coexisting rotating waves in unidirectional rings of bistable Duffing oscillators. *Chaos Interdiscip. J. Nonlinear Sci.* **2023**, *33*, 073126. [CrossRef] [PubMed]
32. Salas, A.; Martínez H, L.J.; Ocampo R, D.L. Analytical and numerical study to a forced van der Pol oscillator. *Math. Probl. Eng.* **2022**, *2022*, 9736427. [CrossRef]
33. Echenausía-Monroy, J.L.; Jafari, S.; Huerta-Cuéllar, G.; Gilardi-Velázquez, H. Predicting the emergence of multistability in a Monoparametric PWL system. *Int. J. Bifurc. Chaos* **2022**, *32*, 2250206. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.